Criteria, which both provide some assurance that the device will resist some physical attacks, but you need to read the small print to determine exactly which threats have been tested.

So far, the mode I've described will always produce the same nonce and the same ciphertext whenever the same plaintext message is encrypted. If you recall from chapter 6, such an encryption scheme is not secure because an attacker can easily tell if the same message has been sent multiple times. For example, if you have a sensor sending packets of data containing sensor readings in a small range of values then an observer may be able to work out what the encrypted sensor readings are after seeing enough of them. This is why normal encryption modes add a unique nonce or random IV in every message: to ensure that different ciphertext is produced even if the same message is encrypted. SIV mode solves this problem by allowing you to include a random IV in the associated data that accompanies the message. Because this associated data is also included in the MAC calculation it ensures that the calculated SIV will be different even if the message is the same. To make this a bit easier, SIV mode allows more than one associated data block to be provided to the cipher; up to 126 blocks in AES-SIV.

Listing 12.16 shows an example of encrypting some data with AES-SIV in Java using an open-source library that implements the mode using AES primitives from Bouncy Castle.[10] To include the library, open the pom.xml file and add the following lines to the dependencies section: